

第 8 讲

Convolution Concept; Constant Cache

目录

1	课程安排与核心内容	1
1.1	核心内容与学习目标	1
1.2	课程安排与阅读材料	1
2	卷积的计算结构与边界处理	2
2.1	邻域加权和与滤波器方向	2
2.2	一维窗口、中心与零填充	2
2.3	二维邻域与图像平滑	4
2.4	直接 CUDA 实现：一个线程计算一个输出	6
3	缓存层次与常量存储器	7
3.1	从主存访问到缓存命中	7
3.2	共享内存、L1/L2 与常量缓存	7
3.3	滤波器的访问一致性	9
3.4	声明、初始化与常量版核函数	9
4	分块的数据复用与三种策略	11
4.1	访问计数、算术强度与复用机会	11
4.2	输出分块、输入覆盖范围与 halo	11
4.3	线程数量与 halo 放置的组合	13
5	分块核函数与索引推导	14
5.1	策略 1：完整输入块与三段载入	14
5.2	策略 1 的连续载入写法	16
5.3	策略 3：核心区缓存与按需 halo 读取	18
6	综合算例与关键结论	19
6.1	末尾不足一个完整块时的计算	19
6.2	带边界条件的精确计数	20
6.3	关键公式与实现关系	21

1 课程安排与核心内容

1.1 核心内容与学习目标

授课日期为 2026 年 9 月 17 日。卷积 (convolution) 是信号、图像和视频处理中的常见并行计算模式，也与科学计算中的模板计算 (stencil computation) 及神经网络有关。学习目标是理解卷积，并利用缓存提高 GPU 数据访问效率。一维与二维邻域加权和确定了输入访问模式，常量缓存与共享内存分别利用系数一致性和输入窗口重叠提高数据复用。

1.2 课程安排与阅读材料

以下安排对应上述授课日期。

事项	安排
Lab 2	授课当周的星期五截止。
Midterm 1 时间	2026 年 10 月 6 日，19:00—22:00。
地点与形式	具体考场分配发布在 Canvas；纸笔考试。
考试范围	Lectures 1—11，Labs 1—4。
时间冲突安排	有有效时间冲突者须在 2026 年 9 月 30 日前邮件联系教师，另行安排考试。
复习与阅读	复习资料位于 Canvas 的 Files/MT1；指定阅读为教材第 7 章。

课程安排与阅读材料。

核心关系

每个输出依赖一个输入邻域，而不同输出使用相同的滤波器。前一种重叠带来输入数据的复用机会，后一种一致性适合常量存储器。分块时，输出覆盖范围确定了需要载入的输入核心区与两侧 halo；线程如何分担这些载入工作，决定了三种策略的区别。

2 卷积的计算结构与边界处理

2.1 邻域加权和与滤波器方向

一个输出元素由其附近输入元素的加权和得到。权重数组称为**卷积掩模**(*convolution mask*)或**滤波器**(*filter*)，通常也称卷积核；为避免与 CUDA 核函数混淆，以下用“滤波器”指权重数组，用“核函数”指设备端并行程序。同一个滤波器通常用于整个输入数组。相邻位置使用不同的输入窗口，但使用同一组权重，因此改变滤波器就能改变对局部信号的处理方式。

连续函数与离散序列的数学卷积分别写为

$$(f * g)(x) = \int_{-\infty}^{\infty} f(\tau)g(x - \tau) d\tau, \quad (f * g)[x] = \sum_{k=-\infty}^{\infty} f[k]g[x - k]. \quad (1)$$

其中 x 指定输出位置， τ 或 k 遍历参与计算的输入位置。表达式中的 $x - \tau$ 、 $x - k$ 对应其中一个函数或序列的翻转与平移。

互相关(*cross-correlation*) 采用不翻转滤波器的滑动匹配。对实值函数，可写为 $(f \star g)(t) = \int f(\tau)g(t + \tau) d\tau$ 。卷积常用于物理系统与滤波，互相关常用于模式匹配与机器学习。CNN 中通常计算互相关，但历史上仍沿用“卷积”的名称；当权重通过训练获得时，是否预先规定翻转可通过学习到的权重方向吸收。

后续数组与 CUDA 示例采用**滤波器按原顺序读取**的约定。这个约定直接决定代码中使用 $M[j]$ ，也便于对应每一个乘加项。给定一组非对称权重时，翻转与不翻转通常会得到不同结果；下面一维示例的权重是对称的，因此该示例的数值无法单独区分这两种方向。

2.2 一维窗口、中心与零填充

输入数组记为 N ，输出数组记为 P ，长度均为 W ；滤波器记为 M ，宽度为 K 。代码使用 `Width`、`Mask_Width` 表示这两个宽度。取奇数 $K = 2r + 1$ ，则半径 $r = (K - 1)/2$ 表示输出中心左右各涉及多少个输入元素；代码中的整数除法 `Mask_Width / 2` 在这个条件下等于 r 。

为形式化说明边界判断，定义零延拓输入 $\tilde{N}[q]$ ：有效下标 $0 \leq q < W$ 时等于 $N[q]$ ，其

他位置等于零。按滤波器原顺序读取，一个输出为

$$P[i] = \sum_{j=0}^{K-1} \tilde{N}[i-r+j]M[j], \quad 0 \leq i < W. \quad (2)$$

这个表达式把三个下标作用分开： i 决定输出位置， j 决定滤波器内的位置， $i-r+j$ 决定当前乘加读取哪个输入。 $j=0$ 对应窗口最左端， $j=r$ 对应输入中心 $N[i]$ ， $j=K-1$ 对应最右端。

以 $N = [1, 2, 3, 4, 5, 6, 7]$ 、 $M = [3, 4, 5, 4, 3]$ 为例， $K = 5$ 、 $r = 2$ 。计算 $P[2]$ 时读取 $N[0]$ 到 $N[4]$ ，而计算 $P[3]$ 时整个窗口向右移动一格：

$$\begin{aligned} P[2] &= 1 \cdot 3 + 2 \cdot 4 + 3 \cdot 5 + 4 \cdot 4 + 5 \cdot 3 = 57, \\ P[3] &= 2 \cdot 3 + 3 \cdot 4 + 4 \cdot 5 + 5 \cdot 4 + 6 \cdot 3 = 76. \end{aligned} \quad (3)$$

每一个输出都需要五次乘加，但这两个窗口共同使用了 $N[1]$ 到 $N[4]$ 。这种重叠正是后续共享内存分块的依据。

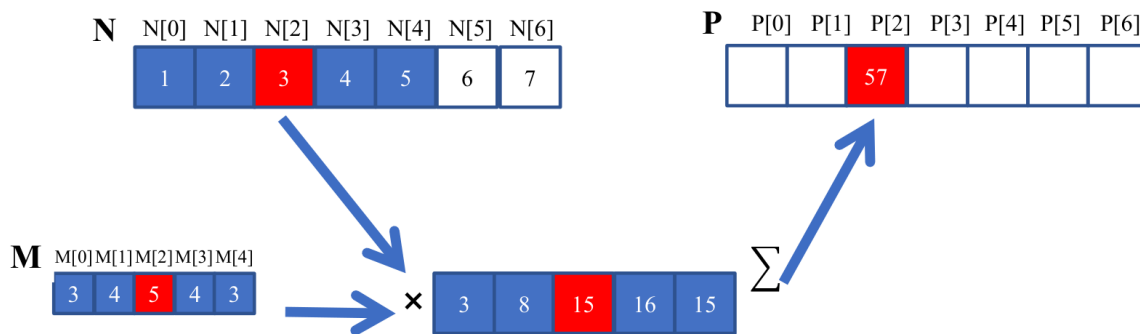


图 1 一维窗口与滤波器逐项相乘，再归约到输出 $P[2] = 57$ 。

当窗口跨出数组边界时，会出现没有对应实际数组元素的位置，称为**幽灵元素 (ghost elements)**。边界规则可以采用零填充，也可以复制边缘值；本讲代码采用零填充。输出 $P[1]$ 的输入窗口为 $[0, 1, 2, 3, 4]$ ，因此 $P[1] = 0 \cdot 3 + 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 4 + 4 \cdot 3 = 38$ 。

零填充可以通过条件判断实现：遇到无效输入下标就跳过乘加。由于该项应为零，跳过与显式读取一个零具有相同的数学作用，因此无须为基本核函数实际分配数组外的元素。按式(2)复算全部位置，可得到 $P = [22, 38, 57, 76, 95, 90, 74]$ ，可作为边界实现的参考结果。

2.3 二维邻域与图像平滑

二维计算将一维窗口扩展为一个矩形邻域。设输入有 H 行、 W 列，滤波器为 $K \times K$ ，两个方向的半径均为 r 。在与一维相同的读取约定下，零延拓输入为 $\tilde{N}$ ，输出行列为 (y, x) ，则

$$P[y, x] = \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} \tilde{N}[y-r+u, x-r+v] M[u, v]. \quad (4)$$

两个求和分别遍历滤波器的行与列；每个输出将整个局部窗口压缩为一个数，输入窗口与滤波器先逐项相乘，再将全部乘积相加。

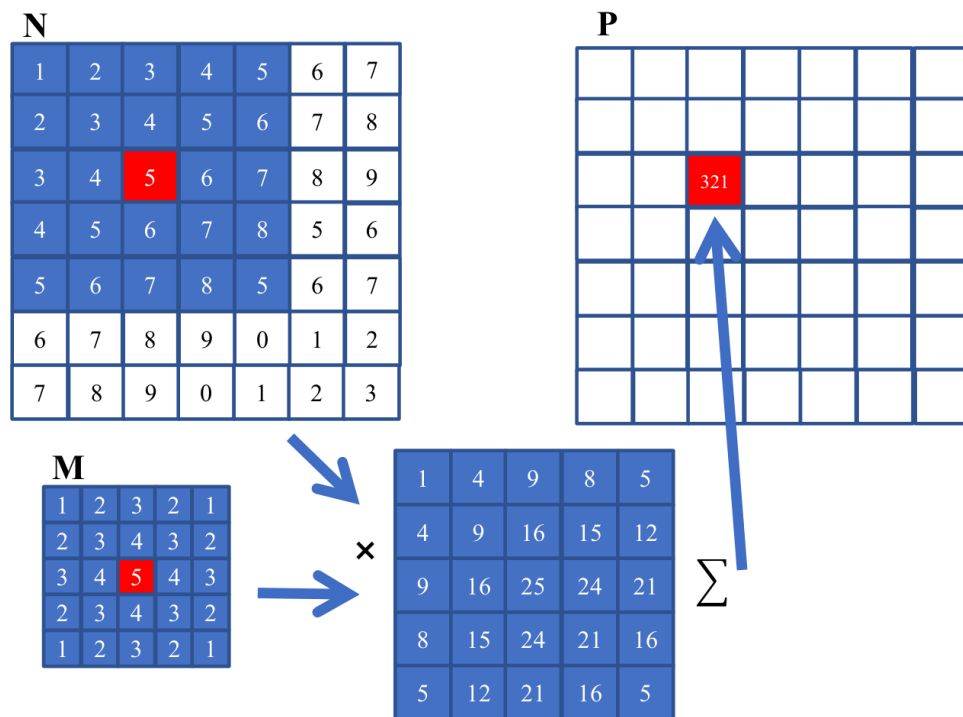


图2 5×5 输入邻域与权重逐项相乘，得到二维内部输出 $P[2,2] = 321$ 。

在图中的零起始坐标下，中心为 $(2,2)$ ，使用输入的前五行、前五列。乘积矩阵各行之和依次为 27, 56, 95, 84, 59，合计 321。在边界例中，输出 $(1,0)$ 的窗口跨过上边界和左边界，填零后五行乘积之和为 0, 16, 34, 34, 28，合计 112。这些行和来自图中数值，可用于把二维索引与实际求和对应起来。

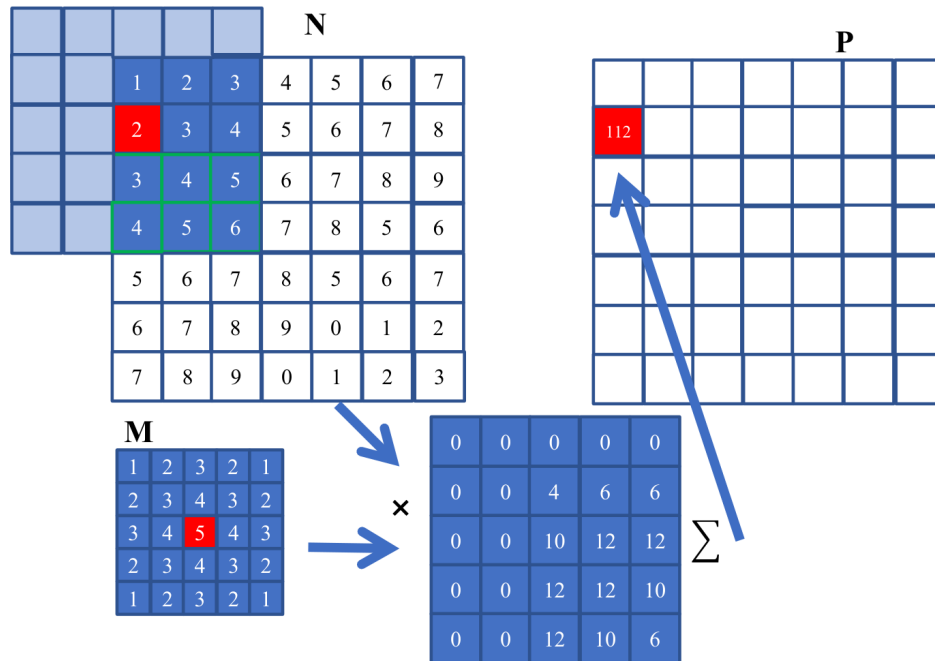


图3 二维零填充：窗口上侧与左侧的无效位置取零，得到边界输出 112。

幽灵单元也称 apron cells 或 halo cells。例如，另一个零填充边界窗口的五行分别为 $[0, 0, 0, 0, 0]$ 、 $[0, 3, 4, 5, 6]$ 、 $[0, 2, 3, 4, 5]$ 、 $[0, 3, 5, 6, 7]$ 、 $[0, 1, 1, 3, 1]$ 。使用图中相同的整数权重，五行乘积之和依次为 0, 52, 54, 61, 12，输出为 179。这些名称都与邻域的外围数据有关；进入分块实现后，需要进一步区分“落在全局数组外的补零位置”与“仍在全局数组内、但落在当前块核心区外的邻域元素”。后一类是真实输入，后续分块必须读取其值。

一个具体的图像滤波器为

$$M = \frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}. \quad (5)$$

平滑作用与边界条件。所有系数非负，矩阵元素之和为 $17 + 66 + 107 + 66 + 17 = 273$ ，归一化后权重和为 1。每个输出是附近像素的加权平均，中心像素的权重较大、远处像素的权重较小，因此它起到平滑、模糊局部变化的作用。若整个窗口内的像素都等于常数 c ，则输出仍为 c 。采用零填充时，图像边缘窗口含有零，因而这一“保持常数”结论限于完整有效窗口；边界值可能被拉低。

2.4 直接 CUDA 实现：一个线程计算一个输出

输出之间只共享输入读取，没有相互依赖的输出写入，所以可让每个线程独立计算一个 $P[i]$ 。一维网格中的全局下标为 $i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$ 。最后一个线程块可能超过数组长度，因此先检查输出下标，再在循环中检查输入下标。

代码 1 直接一维卷积：每个线程计算一个输出，越界输入按零处理。

```

1  __global__ void convolution_1D_basic_kernel(
2      const float* N, const float* M, float* P,
3      int Mask_Width, int Width)
4  {
5      int i = int(blockIdx.x) * int(blockDim.x)
6          + int(threadIdx.x);
7      if (i >= Width) return;
8      float Pvalue = 0.0f;
9      int N_start_point = i - Mask_Width / 2;
10     for (int j = 0; j < Mask_Width; ++j) {
11         int q = N_start_point + j;
12         if (q >= 0 && q < Width) {
13             Pvalue += N[q] * M[j];
14         }
15     }
16     P[i] = Pvalue;
17 }

```

两个范围检查各自保护不同的数组：入口处的 $i \geq \text{Width}$ 保护输出 P ，循环内的检查保护输入 N 。即使 i 合法， $i - r + j$ 仍可能越界，因此两者都需要。局部标量 $Pvalue$ 是每个线程自己的累加器；每个有效线程恰好写一个输出。

实现说明。代码将只读输入写为 `const float*`，并显式把 CUDA 索引转为有符号整数，便于表达减去半径后可能出现的负值。直接版没有共享内存协作，因此无效输出线程可在入口直接返回。分块版的线程还承担输入搬运任务，返回位置需要与这些任务一起分析。

假设每块 B 个线程且 $W > 0$ ，网格块数取 $\lceil W/B \rceil$ 。在整数代码中可以写作 $1 + (\text{Width} - 1) / B$ 。随后，每个线程沿着滤波器执行顺序乘加，线程之间沿输出坐标并行；这同时解释了计算并行性和输入访问的重叠。

3 缓存层次与常量存储器

3.1 从主存访问到缓存命中

GPU 的计算需要持续获得输入。如果每次读取都访问全局存储器中的底层数据，带宽就可能限制执行速度。分块矩阵乘法通过共享内存显式复用输入；**缓存 (cache)** 则在硬件管理下保留近期访问数据，为后续读取提供另一条复用路径。

缓存由若干**缓存行 (cache line)** 组成，每行可存放一段连续地址的数据。当请求的数据尚未在缓存中时，硬件把包含该地址的一行数据带入缓存；缓存中还保存**标签 (tag)**，记录该行对应的内存地址。后续请求找到对应数据时称为缓存命中，否则需要继续访问后面的存储层次。缓存数据是原数据的副本。

连续地址成组传输，使一次读取也可能为后续的邻近读取准备好数据。例如，一次突发传输约 1024 bit，即 128 B 时，相邻数据可以随请求位置一起进入缓存。这个示例以一次突发对应一行说明成组搬运；具体缓存行与事务大小取决于存储层次和硬件实现。

空间局部性与时间局部性。空间局部性 (*spatial locality*) 指相近的时间里访问连续或邻近地址；时间局部性 (*temporal locality*) 指一个数据在短时间内被多次访问。对于固定线程，一维卷积沿着窗口依次读取 $N[i-r], N[i-r+1], \dots$ ，体现空间局部性；相邻输出窗口重叠，使一个输入可能在较短时间内再次被使用，体现时间局部性。两种局部性都能提高缓存发挥作用的机会。

缓存容量小于全局存储器，不能保留全部输入。当需要为新缓存行腾出空间时，已有行会被替换；一种典型的替换思路是**最近最少使用 (least recently used, LRU)**，即优先替换最久没有被访问的缓存行。能否命中还取决于相关数据在再次访问前是否仍留在缓存中，因此“存在复用机会”与“每次都能缓存命中”需要分开判断。

3.2 共享内存、L1/L2 与常量缓存

CUDA 的存储空间在可见范围、读写方式及用途上不同。下表给出用于直观比较的近似访问周期。实际延迟取决于器件、缓存命中及访问模式，这些示例量级不构成统一的运行时间预测模型。

存储空间	可见范围	线程权限	示例访问周期
寄存器	单线程	读、写	~ 1 cycle
共享内存	同一线程块	读、写	~ 5 cycles
全局存储器	网格内线程	读、写	~ 500 cycles
常量存储器	网格内线程	只读	缓存条件下 ~ 5 cycles

表 1 CUDA 存储空间的程序员视图与教学访问量级。

共享内存与缓存都利用靠近 SM 的片上存储减少主存压力，但**内容由谁管理**决定了它们的编程方式。共享内存是程序显式管理的暂存区 (scratchpad)：程序决定把什么值写入哪个位置，并在需要时同步线程。缓存的内容则由微体系结构根据内存访问自动选择和替换，程序通常不显式指定某个数据应放入哪一条缓存行。共享内存还可以保存中间结果，不要求内容始终对应全局存储器中的一个副本。

在 Volta 及之后的相关设计中，共享内存和 L1 可以使用统一的物理资源，并在这些用途之间分配。物理资源可以共享，访问与管理接口仍保持各自特点。图4中的局部片上存储靠近各个 SM，L2 位于多个 SM 与全局存储器之间。

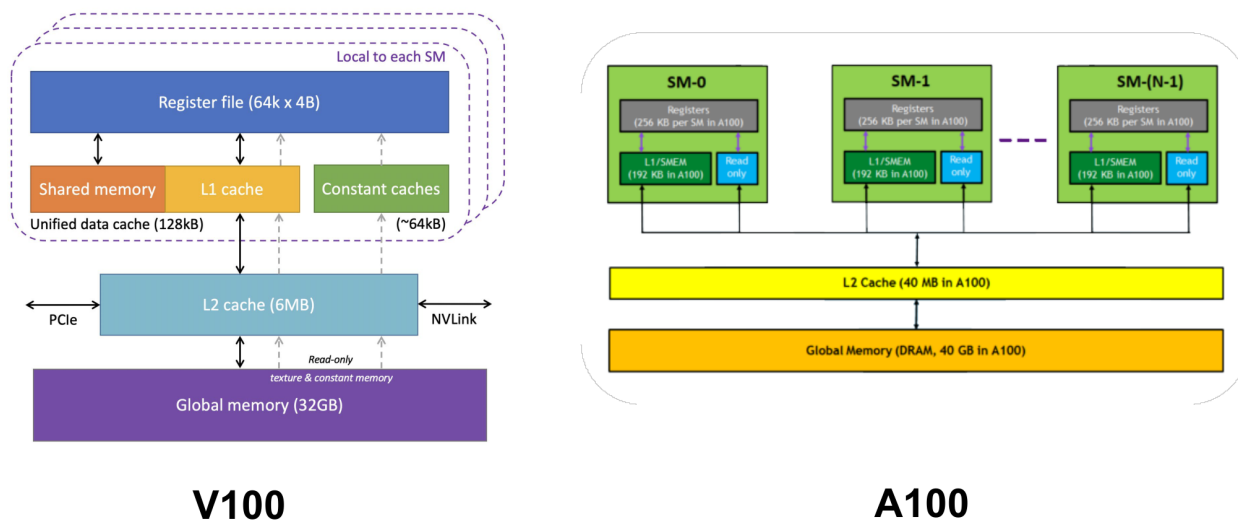


图 4 V100 与 A100 的缓存层次示例：SM 附近的局部存储、共享 L2 及其后的全局存储器。图中容量保留对应示例器件的标注。

对于可修改的缓存数据，硬件需要处理修改状态及与后备内存的一致性。**常量缓存 (constant cache)** 服务于核函数执行期间保持不变的数据，读取路径可以针对这类模式优化。常量和纹理的只读访问路径，与一般全局访问使用的 L1 路径具有不同用途。滤波器系数的实现使用常量存储器。

3.3 滤波器的访问一致性

滤波器 M 有三个相关性质：计算所有输出都需要它，它在一次网格执行期间保持不变，而且各线程按照相同的顺序读取系数。在循环第 j 次迭代，输出位置不同的线程读取各自的 $N[i - r + j]$ ，同时读取同一个 $M[j]$ 。小而重复使用、且访问顺序一致的滤波器因此适合放入常量存储器 (constant memory)。

同一轮乘加的两种地址关系

对相邻输出线程 $i, i + 1, \dots$ ，固定 j 时，输入地址为 $i - r + j, i + 1 - r + j, \dots$ ，沿数组相邻分布；滤波器地址均为 j 。输入复用与系数复用具有不同结构，分别对应后续共享内存分块与常量存储器优化。

同地址访问与广播。同一 warp 的线程访问同一常量地址时可以共享该次读取；访问多个不同地址时，请求按不同地址串行处理，代价随不同地址数增加。因此 $Mc[j]$ 的一致访问有利，而按线程编号访问各不相同的常量位置就不具备同样的优势。

输入 N 在本次卷积中也保持只读，但往往比滤波器大得多。常量存储器总容量上限为 64 KB，因此是否只读只是选择条件之一；还要考虑容量与访问结构。64 KB 指常量地址空间的容量约束，不能从这个数字直接推出每个 SM 的常量缓存大小。常量缓存未命中时仍需取得后备数据。

3.4 声明、初始化与常量版核函数

常量数组通过 `__constant__` 在所有主机函数与设备核函数之外声明。主机用 `cudaMemcpyToSymbol` 将权重复制到这个符号；主机端数据可以运行时生成，设备端执行网格期间按只读方式使用这些系数。

代码 2 常量滤波器声明及主机初始化；五个系数沿用一维数值例。

```

1 constexpr int MASK_WIDTH = 5;
2 __constant__ float Mc[MASK_WIDTH];
3 // Inside a host function, before launching the kernel:
4 float Mask[MASK_WIDTH] = {3, 4, 5, 4, 3};
5 cudaError_t status = cudaMemcpyToSymbol(
6     Mc, Mask, MASK_WIDTH * sizeof(float));
7 if (status != cudaSuccess) {
8     // Report or propagate the error; do not launch on failure.
9     return;
10 }
```

复制长度的单位是字节，所以必须乘以 `sizeof(float)`。这次调用采用默认偏移 0 与

默认方向 `cudaMemcpyHostToDevice`。权重也可以由随机数生成器产生；核函数启动之前，都需要先完成权重初始化和复制。

常量版只把原先的 `M[j]` 改为 `Mc[j]`，窗口起点、边界条件与求和次序均保持不变：

代码 3 使用常量滤波器的一维卷积。

```

1  __global__ void convolution_1D_constant_kernel(
2      const float* N, float* P, int Mask_Width, int Width)
3  {
4      int i = int(blockIdx.x) * int(blockDim.x)
5              + int(threadIdx.x);
6      if (i >= Width) return;
7      float Pvalue = 0.0f;
8      int N_start_point = i - Mask_Width / 2;
9      for (int j = 0; j < Mask_Width; ++j) {
10         int q = N_start_point + j;
11         if (q >= 0 && q < Width) {
12             Pvalue += N[q] * Mc[j];
13         }
14     }
15     P[i] = Pvalue;
16 }

```

参数与初始化条件。常量版直接读取设备符号 `Mc[j]`，因此核函数参数只包含输入、输出及两种宽度，无须传入滤波器指针。调用应传入已经初始化的有效宽度，满足 $0 < K \leq \text{MASK_WIDTH}$ ；后续完整示例固定采用奇数 $K = 5$ 。

假设 `Nd`、`Pd` 已完成设备端分配、`Nd` 已存有输入， $W > 0$ ，取每块 B 个线程，则调用形式为

代码 4 与常量版核函数匹配的启动参数。

```

1  dim3 dimBlock(B);
2  dim3 dimGrid(1 + (Width - 1) / B);
3  convolution_1D_constant_kernel<<<dimGrid, dimBlock>>>>(
4      Nd, Pd, MASK_WIDTH, Width);

```

常量版改善了系数的读取路径，但相邻线程仍会反复请求重叠的输入窗口。共享内存分块进一步利用 N 的复用。

4 分块的数据复用与三种策略

4.1 访问计数、算术强度与复用机会

对于不接触边界的一个一维输出，直接实现读取 K 个输入与 K 个系数，共 $2K$ 次标量读取，并执行 K 次乘法和 K 次加法，共 $2K$ 次浮点运算。二维 $K \times K$ 滤波器对应 $2K^2$ 次标量读取与 $2K^2$ 次浮点运算。因此增加滤波器宽度时，直接算法的读取量和计算量一起增长，其复用前的算术强度较低。

算术强度 (arithmetic intensity) 用计算量与数据搬运量之比描述计算对带宽的需求。按每个 `float` 为 4 B，将标量读取次数换成字节：一维内部输出读入 $8K$ B、写出 4 B。忽略复用时，仅以读入量为分母得到 $2K/(8K) = 0.25$ FLOP/B；把输出写入也计入，得到

$$I_{\text{direct,1D}} = \frac{2K}{8K + 4}, \quad I_{\text{direct,2D}} = \frac{2K^2}{8K^2 + 4}. \quad (6)$$

较低的算术强度说明直接算法容易受到带宽限制；常量缓存与共享内存的作用，就是减少同一份数据在较慢存储层次上的重复获取。

计数口径。这里的“读取次数”首先指代码层面的标量访问。缓存命中、常量同地址访问以及成组内存事务会改变实际 DRAM 流量，所以不能直接把 $2K$ 次读取等同于 $2K$ 次独立 DRAM 事务。边界线程又会在直接核函数中跳过无效项，此时实际执行的读取与乘加比内部输出少。后面的精确例题始终先说明统计的是输入请求、字节还是乘加次数。

4.2 输出分块、输入覆盖范围与 halo

设一个线程块生成 B 个连续输出，块号为 b ，输出起点为 $s = bB$ 。输出范围是 $[s, s+B-1]$ 。最左输出需要从 $s-r$ 开始读取，最右输出需要读到 $s+B-1+r$ ，因此整个块需要的输入范围为

$$[s-r, s+B+r-1], \quad T_{\text{in}} = B + 2r = B + K - 1. \quad (7)$$

这里 B 是输出块宽度， T_{in} 是包含边缘邻域的输入块宽度。输入核心区与输出对齐，左右各多出的 r 个位置称为**晕区 (halo)**。

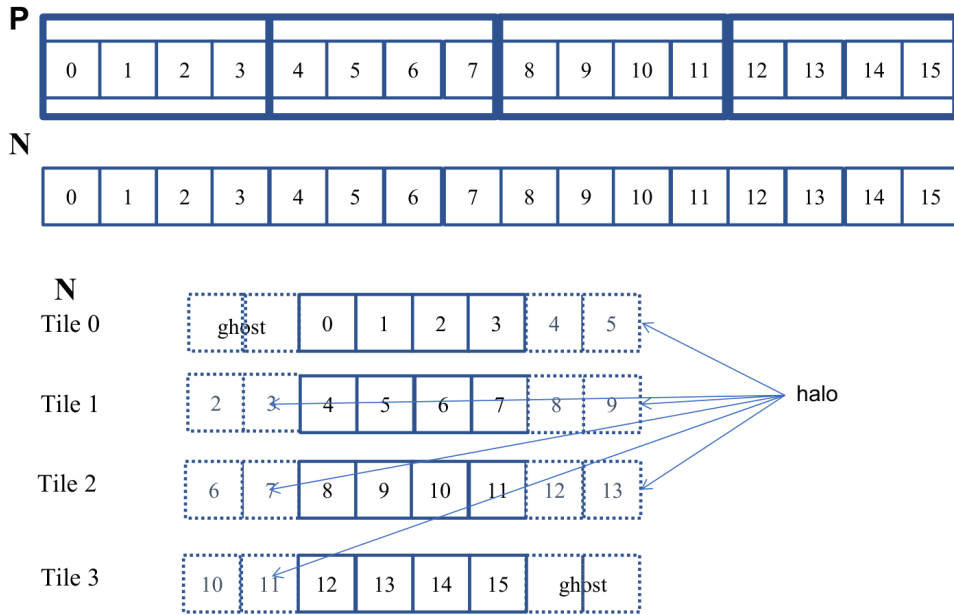


图 5 $W = 16$ 、 $B = 4$ 、 $K = 5$ 时的四个输出块及其输入邻域。中间块的 halo 是真实输入，首末块还包含越界的幽灵位置。

以 $b = 1$ 为例，这个块生成 $P[4]$ 到 $P[7]$ ，输入需要 $N[2]$ 到 $N[9]$ 。核心区是 $N[4 : 7]$ ，左 halo 是 $N[2 : 3]$ ，右 halo 是 $N[8 : 9]$ 。相邻块的输出不重叠、输入可以重叠，所以线程块可以独立写各自的输出，但边缘输入可能被多个块读取。第一个块需要的 $N[-2]$ 与 $N[-1]$ 才属于全局数组外的幽灵位置，按边界规则填零。

块内复用的数量。对于 $B = 4, K = 5$ ，四个输出分别需要五个输入，共产生 20 次输入使用，但覆盖的不同输入只有八个。 $N[2]$ 到 $N[9]$ 的块内使用次数依次为 1, 2, 3, 4, 4, 3, 2, 1，总和为 20。一旦把这八个值放入共享内存，块内后续使用就可以从共享内存读取。

对于远离全局边界的完整块，直接方法的输入请求数为 BK ；把完整输入块载入一次则为 $B + 2r$ 。因此只比较 N 的块内读取请求，可写出复用因子

$$R_N = \frac{BK}{B + 2r} = \frac{BK}{B + K - 1}. \quad (8)$$

这给出示例中的 $20/8 = 2.5$ 。固定 K 而增大 B 时，halo 相对占比 $2r/B$ 下降， R_N 趋近于 K 。这些是输入覆盖关系推得的请求量比较，没有计入系数缓存、同步或硬件资源对实际速度的影响。

4.3 线程数量与 halo 放置的组合

分块之后仍有两个决定：线程数与输出块匹配，还是与输入块匹配；halo 是否也放入共享内存。由此形成三种主要策略。

比较项	策略 1	策略 2	策略 3
线程块覆盖	B 个输出	$B + 2r$ 个输入	B 个输出
每块线程数	B	$B + 2r$	B
共享内存内容	核心区与两侧 halo	核心区与两侧 halo	仅核心区
载入方式	部分线程额外载入	每线程载入一个输入	每线程载入一个核心输入
计算方式	每线程计算一个输出	仅部分线程计算输出	每线程计算一个输出
计算时 halo	从共享内存读取	从共享内存读取	从全局地址读取
主要取舍	载入阶段任务不均	计算阶段部分线程闲置	计算循环选择两种读取路径

表 2 以相同的 B 个输出为比较对象，线程数与输入存放方式的差异。

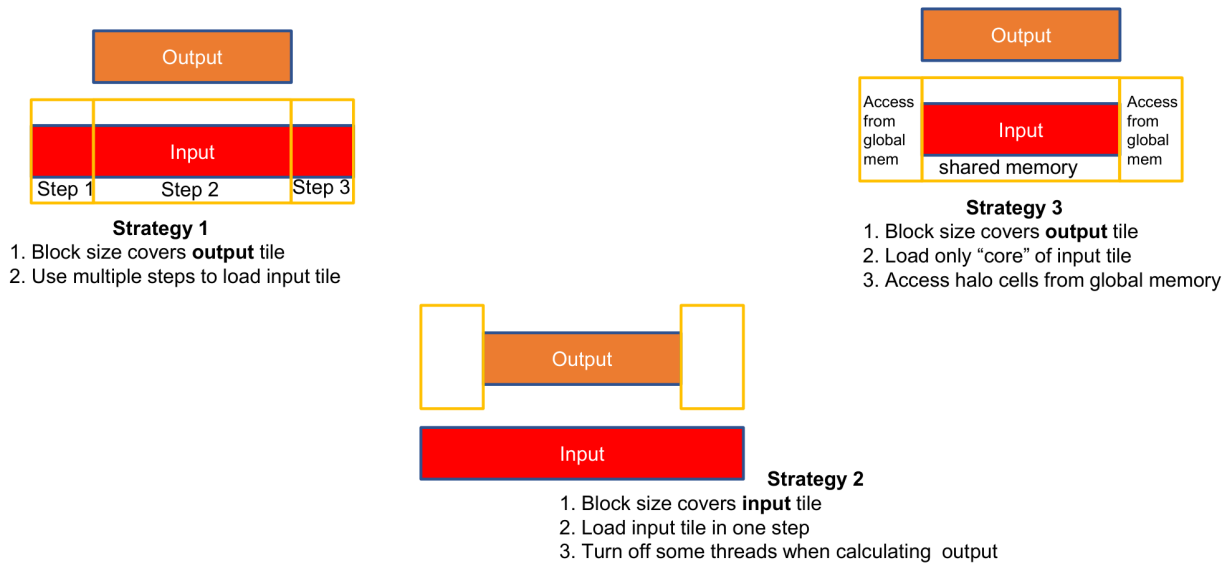


图 6 三种分块策略的输入、输出和线程工作范围。策略 1 分步载入完整输入块；策略 2 为输入配置线程；策略 3 将 halo 留给全局读取路径。

策略 1 的取舍。将 halo 一并载入共享内存后，卷积计算循环的输入都来自同一种位置，不需要在每次乘加时选择共享或全局路径。连续位置可以按组载入，也有利于合并访问。代价是共享内存由 B 个元素增加到 $B + 2r$ 个元素，而且部分线程在载入阶段必须多做工作。载入任务按线程编号分配时，可能出现部分线程执行分支、其他线程不执行的情况。

策略 3 的取舍。只缓存核心区，把共享内存优先用于块内复用较高的输入。计算每一个乘加项时判断输入是否位于本块核心区：位于核心区就读共享内存，位于 halo 就按全局地

址读取。同一 warp 内的线程可能选择不同路径；窄 halo 的访问也可能只利用一次成组内存传输的一小部分。

按 $r \leq B$ 的完整内部块展开计数，左侧各输出对左 halo 的读取数为 $r, r-1, \dots, 1$ ，右侧对称，因此两侧一共 $r(r+1)$ 次全局 halo 请求。再加 B 次核心区载入，策略 3 对 N 的请求数为 $B + r(r+1)$ 。在 $B = 4, r = 2$ 的例子中，这个数为 10，策略 1 则是 8。该比较按程序请求计数；策略 3 的 halo 请求可能由缓存满足，不能只靠这两个数判定实测速度。

策略 2 的取舍。让每个线程载入一个输入，载入阶段无需由少量线程额外承担 halo；随后仅有与输出对应的线程执行乘加。按相同输出宽度 B 比较，计算输出的线程比例为 $B/(B+2r)$ 。这种分配减少高延迟载入阶段的工作分歧，同时让部分线程不参与较低延迟的计算阶段。全局数组首末处的边界填零仍然需要处理。

5 分块核函数与索引推导

5.1 策略 1：完整输入块与三段载入

策略 1 为一个输出块配置 B 个线程。局部线程号记为 t ，块起点为 $s = bB$ ，线程的输出下标为 $i = s + t$ 。共享数组 N_ds 需要容纳 $B + 2r$ 个元素。在这个共享数组中，最左位置从整个输入窗口的左端开始，因而有统一映射

$$N_{ds}[q] = \tilde{N}[s - r + q], \quad 0 \leq q < B + 2r. \quad (9)$$

右侧的 $\tilde{N}$ 已包含零填充约定。这个关系同时描述了左 halo、核心区与右 halo，是核对所有搬运代码的共同依据。

线程 t 计算的输出为 $P[s + t]$ ，其第 j 项需要 $\tilde{N}[s + t - r + j]$ 。代入式(9)可知，该项的共享内存下标恰好是 $q = t + j$ 。因此乘加使用 $N_ds[threadIdx.x + j]$ ，不再额外减去半径：半径偏移已经包含在共享数组起点的选择中。

三段载入的分工。左 halo 由局部编号最大的 r 个线程搬运，即 $t \geq B - r$ 的线程。其全局下标为 $(b-1)B + t = s - B + t$ ，共享下标为 $t - (B - r)$ 。当 $t = B - r$ 时，这两个下标分别为 $s - r$ 与 0；当 $t = B - 1$ 时，分别为 $s - 1$ 与 $r - 1$ ，正好覆盖左 halo。

所有线程将核心值 $N[s + t]$ 放入共享位置 $r + t$ 。右 halo 则由局部编号最小的 r 个线程搬运：全局下标为 $(b+1)B + t = s + B + t$ ，共享下标为 $r + B + t$ 。三个目标区间 $[0, r-1]$ 、 $[r, r+B-1]$ 、 $[r+B, r+B+r-1]$ 连续且不重叠，共同填满 $B + 2r$ 个位置。

载入区域	负责的局部线程 t	全局输入下标	共享数组下标
左 halo	$B - r, \dots, B - 1$	$s - B + t$	$t - (B - r)$
核心区	$0, \dots, B - 1$	$s + t$	$r + t$
右 halo	$0, \dots, r - 1$	$s + B + t$	$r + B + t$

表 3 策略 1 的三段搬运映射；要求 $r \leq B$ ，并对无效全局下标填零。

在 $b = 1, B = 4, r = 2$ 的例子中，线程 2,3 把 $N[2], N[3]$ 放入共享位置 0,1；所有线程把 $N[4], \dots, N[7]$ 放入位置 2, ..., 5；线程 0,1 把 $N[8], N[9]$ 放入位置 6,7。相同线程可以同时承担核心区和 halo 的载入，且这些工作写入不同的共享地址。

完整的三段载入核函数如下。滤波器存放在常量数组 **Mc** 中，宽度固定为 $K = 5$ ，由 **MASK_WIDTH** 指定。块宽 B 为模板参数，启动时每块必须恰好配置 B 个线程。共享数组在核函数内声明；有符号索引与双侧范围检查保证左右边界按零填充处理。

代码 5 策略 1：核心区与两侧 halo 分段载入。

```

1  template<int B>
2  __global__ void convolution_1D_tiled_three_loads(
3      const float* N, float* P, int Width)
4  {
5      constexpr int radius = MASK_WIDTH / 2;
6      static_assert(MASK_WIDTH % 2 == 1, "Odd mask required");
7      static_assert(B >= radius, "B must cover each halo");
8      __shared__ float N_ds[B + MASK_WIDTH - 1];
9      int t = int(threadIdx.x);
10     int s = int(blockIdx.x) * B;
11     int i = s + t;
12     if (t >= B - radius) {
13         int q = s - B + t;
14         N_ds[t - (B - radius)] =
15             (0 <= q && q < Width) ? N[q] : 0.0f;
16     }
17     N_ds[radius + t] = (i < Width) ? N[i] : 0.0f;
18     if (t < radius) {
19         int q = s + B + t;
20         N_ds[radius + B + t] =
21             (0 <= q && q < Width) ? N[q] : 0.0f;
22     }
23     __syncthreads();
24     float Pvalue = 0.0f;
25     for (int j = 0; j < MASK_WIDTH; ++j) {
26         Pvalue += N_ds[t + j] * Mc[j];
27     }
28     if (i < Width) P[i] = Pvalue;
29 }

```

为何载入后需要同步。一个线程计算时会读取其他线程写入的共享位置。屏障 `__syncthreads()` 将共同载入与后续读取分开，使参与线程在开始乘加前可见已完成的共享内存写入。三段载入本身写入互不重叠的位置，中间不需要互相消费刚写出的数据，因此一次位于载入之后的屏障即可。

输出越界不等于搬运任务消失。最后一个块中，某个线程的 i 即使超过 $W - 1$ ，它负责的共享位置仍可能被其他有效输出使用，包括对幽灵位置写零。因此完整代码先让所有线程完成分工、经过屏障，再用 $i < W$ 保护写出。直接版入口处的提前返回不能原样搬到这段代码的载入前；具体缺失哪些位置，可在第6节的末尾块例子中核对。

代码采用“全部线程计算、只保护输出写入”的结构，所以末尾无有效输出的线程也会执行乘加。若将计算限制为有效输出线程，需要把这一限制放在共同载入与屏障之后，且不得跳过仍被其他线程需要的搬运任务。

5.2 策略 1 的连续载入写法

三段写法按“左、核心、右”分配输入。另一种写法直接把完整共享数组看成一段连续的长度 $B + 2r$ 的区间：第一轮由所有线程填入前 B 个位置，第二轮由部分线程填入剩余 $2r = K - 1$ 个位置。两种写法最终都满足式(9)，所以计算循环完全相同。

第一轮让线程 t 读取 `start = i - radius`，写入 `N_ds[t]`。第二轮令 `start += B`，由 $t < 2r$ 的线程读取该地址，写入 `N_ds[B+t]`。这里第二轮补齐的是“完整输入块的末尾”，其中可能同时包含核心区的一部分和右 halo。

例如 $B = 4, r = 2, s = 4$ 时，第一轮载入 $N[2 : 5]$ ，即左 halo 加核心区的前半部分；第二轮载入 $N[6 : 9]$ ，即核心区后半部分加右 halo。与三段写法相比，区别在于每个线程负责哪些搬运，而非最终共享数组的内容。

代码 6 策略 1: 两轮连续载入; 需要 $K-1 \leq B$ 。

```

1  template<int B>
2  __global__ void convolution_1D_tiled_two_loads(
3      const float* N, float* P, int Width)
4  {
5      constexpr int radius = MASK_WIDTH / 2;
6      static_assert(MASK_WIDTH % 2 == 1, "Odd mask required");
7      static_assert(B >= MASK_WIDTH - 1, "Two loads need B >= K-1");
8      __shared__ float N_ds[B + MASK_WIDTH - 1];
9      int t = int(threadIdx.x);
10     int i = int(blockIdx.x) * B + t;
11     int start = i - radius;
12     N_ds[t] = (0 <= start && start < Width)
13         ? N[start] : 0.0f;
14     if (t < MASK_WIDTH - 1) {
15         start += B;
16         N_ds[t + B] = (0 <= start && start < Width)
17             ? N[start] : 0.0f;
18     }
19     __syncthreads();
20     float Pvalue = 0.0f;
21     for (int j = 0; j < MASK_WIDTH; ++j) {
22         Pvalue += N_ds[t + j] * Mc[j];
23     }
24     if (i < Width) P[i] = Pvalue;
25 }

```

两轮载入的覆盖条件。第二轮最多有 B 个线程各填一个位置，而剩余位置数为 $2r$ ，因此要求 $2r \leq B$ 。条件 $MASK_WIDTH - 1 > threadIdx.x$ 在这个条件下可以覆盖所有剩余位置；当 $K-1 > B$ 时，仅有两轮就无法填满共享数组。这个限制由载入分工推得，不影响卷积计算公式本身。

由同一映射导出的循环形式。需要支持更多载入轮次时，让线程 t 依次负责 $q = t, t + B, t + 2B, \dots$ ，直到 $q \geq B + K - 1$ 。每个合法共享下标都有唯一的模 B 余数，因此恰好由一个线程负责：

代码 7 连续载入的索引推广；替换载入阶段，之后仍需相同屏障与计算循环。

```

1  int s = int(blockIdx.x) * B;
2  for (int q = t; q < B + MASK_WIDTH - 1; q += B) {
3      int g = s - radius + q;
4      N_ds[q] = (0 <= g && g < Width) ? N[g] : 0.0f;
5  }
6  __syncthreads();

```

该循环将两轮载入推广到任意所需轮数。当输入块比线程块更宽时，各线程以步长 B 分摊载入任务，仍保持共享位置恰好初始化一次。

5.3 策略 3: 核心区缓存与按需 halo 读取

策略 3 的共享数组只容纳 B 个核心输入。线程 t 将 $N[s+t]$ 放入 $N_ds[t]$, 所以共享映射变为 $N_{ds}[q] = \tilde{N}[s+q]$, $0 \leq q < B$ 。载入完成后经过屏障, 再进入卷积循环。

对输出 $i = s+t$ 的第 j 项, 输入全局下标为 $g = i - r + j$ 。先判断它是否属于全局有效区间 $[0, W)$; 有效时再判断是否属于本块核心区 $[s, s+B)$ 。位于核心区的输入, 其共享下标为 $g - s = t - r + j$; 位于 halo 的输入直接使用全局下标 g 读取。

代码 8 策略 3: 缓存核心区, 并检查全局输入范围。

```

1  template<int B>
2  __global__ void convolution_1D_core_shared(
3      const float* N, float* P, int Width)
4  {
5      constexpr int radius = MASK_WIDTH / 2;
6      static_assert(MASK_WIDTH % 2 == 1, "Odd mask required");
7      __shared__ float N_ds[B];
8      int t = int(threadIdx.x);
9      int s = int(blockIdx.x) * B;
10     int i = s + t;
11     N_ds[t] = (i < Width) ? N[i] : 0.0f;
12     __syncthreads();
13     float Pvalue = 0.0f;
14     for (int j = 0; j < MASK_WIDTH; ++j) {
15         int N_index = i - radius + j;
16         if (0 <= N_index && N_index < Width) {
17             if (s <= N_index && N_index < s + B) {
18                 Pvalue += N_ds[t - radius + j] * Mc[j];
19             } else {
20                 Pvalue += N[N_index] * Mc[j];
21             }
22         }
23     }
24     if (i < Width) P[i] = Pvalue;
25 }

```

共享下标安全性。进入共享读取分支时, 已有 $s \leq g < s+B$, 因此 $0 \leq g-s < B$, 等价于 $0 \leq t-r+j < B$ 。尽管表达式中含有减去半径, 分支条件保证实际读取时下标合法。外层条件负责全局数组边界, 内层条件负责选择数据来源, 两者不能相互替代。

以 $s=4, B=4, r=2$ 的线程 $t=0$ 为例, 它计算 $P[4]$, 依次读取 $N[2]$ 到 $N[6]$ 。前两项不属于核心区, 从全局地址读取; 后三项属于核心区, 从共享位置 0,1,2 读取。与此同时, 计算 $P[6]$ 的线程可能在相同的循环迭代已经使用共享内存, 因而同一 warp 内可能出现不同读取分支。

两种共享数组起点决定两种计算下标

策略 1 的共享位置 0 对应全局位置 $s-r$ ，因此读取下标为 $t+j$ 。策略 3 的共享位置 0 对应全局位置 s ，因此读取下标为 $t-r+j$ ，且仅在输入属于核心区时使用。这一差异完全来自共享数组表示的输入范围。

核心区载入同样需要检查边界。 $(i < \text{Width}) ? N[i] : 0.0f$ 在第一次全局读取之前处理了末尾块；仅在最后给 $P[i]$ 加条件，并不能保护此前已经执行的 $N[i]$ 读取。策略 3 与策略 1 都要完成共同载入和屏障，随后才进入各自的计算路径。

6 综合算例与关键结论

6.1 末尾不足一个完整块时的计算

沿用一维输入和滤波器，逐项分析末尾块的载入与计算。取 $W = 7$ 、 $B = 4$ 、 $K = 5$ 、 $r = 2$ ，输入 $N = [1, 2, 3, 4, 5, 6, 7]$ ，滤波器 $M = [3, 4, 5, 4, 3]$ 。网格需要两块，分别负责输出下标 0—3 与 4—7，其中 $P[7]$ 没有对应的有效输出位置。

第二块从 $s = 4$ 开始。策略 1 的共享数组覆盖全局位置 2 到 9，依次填入 $[3, 4, 5, 6, 7, 0, 0, 0]$ 。前五个值来自真实输入，后三个值按零填充规则生成。三段载入中，线程 $t = 3$ 虽然对应无效输出 $i = 7$ ，仍负责把 $N[3] = 4$ 放入共享位置 1，并把其核心位置写为零。

如果线程 $t = 3$ 在搬运前提前返回，共享位置 1 就没有被它初始化。有效线程 $t = 0$ 计算 $P[4]$ 时需要共享位置 0 到 4，其中就包含这一位置。因此有效输出会依赖未完成的搬运；只查看“哪些线程要写输出”无法判断哪些线程可以提前结束。

局部线程	输出位置	策略 1 读取的共享值	结果与写入
$t = 0$	$P[4]$	$[3, 4, 5, 6, 7]$	$9 + 16 + 25 + 24 + 21 = 95$
$t = 1$	$P[5]$	$[4, 5, 6, 7, 0]$	$12 + 20 + 30 + 28 = 90$
$t = 2$	$P[6]$	$[5, 6, 7, 0, 0]$	$15 + 24 + 35 = 74$
$t = 3$	无有效输出	$[6, 7, 0, 0, 0]$	可执行乘加，但不写 $P[7]$

表 4 末尾块的完整窗口；零值也占据明确的共享位置。

两轮载入写法中，第二块第一轮载入 $[N[2], N[3], N[4], N[5]]$ ，第二轮载入 $[N[6], 0, 0, 0]$ ，最终共享内容与三段写法相同。在不同写法中负责同一数据的线程可能不同，但只要式(9)成立，乘加结果就一致。

对策略 3，同一块的共享内容只有 $[N[4], N[5], N[6], 0] = [5, 6, 7, 0]$ 。计算 $P[4]$ 时从全局地址取得 $N[2], N[3]$ ，从共享数组取得 $N[4], N[5], N[6]$ 。不同数据放置方式由不同下标转换

衔接到同一个卷积窗口。

6.2 带边界条件的精确计数

对直接核函数，位置 i 的有效乘加项数就是窗口 $[i-r, i+r]$ 与数组 $[0, W-1]$ 的交集长度。将边界判断写成计数式，可得

$$v_i = \min(W-1, i+r) - \max(0, i-r) + 1, \quad 0 \leq i < W. \quad (10)$$

对于 $W=7, r=2$, v_i 依次为 3, 4, 5, 5, 5, 4, 3，总计 29。直接核函数每个有效项读一个输入和一个系数，因此一共执行 58 次标量读取，对应 232 B 的逻辑读入；有效输出写入 7 个 **float**，对应 28 B；乘法加法合计为 58 FLOPs。这些数字来自代码实际通过边界分支的项数。

若只比较策略 1 对输入 N 的请求，首块的输入覆盖 $[-2, 5]$ ，其中六个位置有效；第二块覆盖 $[2, 9]$ ，其中五个位置有效，共读入 11 个真实输入值，即 44 B。共享内存中总共写入 $2 \times 8 = 16$ 个位置，其余五个位置由线程直接写零。常量系数的读取和缓存事务没有包含在这 44 B 中。

完整输入块已在载入阶段填零，策略 1 的乘加循环就不再跳过零项。七个有效输出各执行五次乘加，需要 70 FLOPs；代码 5 与代码 6 采用全部线程计算的结构，八个已启动线程共执行 80 FLOPs，最后一个线程的结果不写出。减少输入请求可以同时增加零项或无效输出上的算术工作，这正是按计数对象分别核对的意义。

再考虑 $W=16, B=4, K=5$ 的布局。四个块的有效输入请求数依次为 6, 8, 8, 6，策略 1 合计请求 28 个输入值。直接核函数的有效输入使用次数为 $3+4+12 \times 5+4+3=74$ 。该例按整个网格统计的输入请求减少因子为 $74/28 \approx 2.64$ ，而式(8)的 2.5 比较的是一个远离边界的完整内部块。两者采用了不同的统计范围。

6.3 关键公式与实现关系

卷积计算、缓存与分块可以通过以下关系一起核对。公式中的 $K = 2r + 1$, B 表示每块输出数, $s = bB$ 表示输出块起点; 策略 2 若按输入块大小配置线程, 线程数应相应增加为 $B + 2r$ 。

对象	关系
输出与输入窗口	$P[i] = \sum_{j=0}^{K-1} \tilde{N}[i - r + j]M[j]$
一块所需输入数	$T_{in} = B + 2r = B + K - 1$
完整输入块的共享映射	$N_{ds}[q] = \tilde{N}[s - r + q]$
策略 1 的计算下标	$q = t + j$
核心区的共享映射	$N_{ds}[q] = \tilde{N}[s + q]$
策略 3 的共享计算下标	$q = t - r + j$, 仅当 $s \leq i - r + j < s + B$
内部块输入请求比较	直接法 BK ; 策略 1 为 $B + 2r$
策略 1 的线程覆盖条件	三段载入要求 $r \leq B$; 两轮载入要求 $2r \leq B$
共享内存容量	策略 1、2 为 $4(B + 2r)B$; 策略 3 为 $4B B$

表 5 卷积窗口、共享索引与资源用量。

从输出窗口出发确定完整输入范围, 再选择共享数组的起点, 即可推得载入与计算下标。常量存储器处理统一系数的读取, 共享内存处理邻域输入的复用; 二者连接到同一个卷积计算, 而优化的对象各不相同。